Day 3 – DRF Basics: Game & Publisher APIs

Goals: Expose the models via REST endpoints using DRF ViewSets and a router.

Add to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'games',
]
```

games/serializers.py:

```
from rest_framework import serializers
from .models import Game, Publisher


class GameSerializer(serializers.ModelSerializer):
    class Meta:
        model = Game
        fields = '__all__'


class PublisherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Publisher
        exclude = ('webhook_secret',)  # never expose the secret
```

games/viewsets.py:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer


class GameViewSet(viewsets.ModelViewSet):
    queryset = Game.objects.all()
    serializer_class = GameSerializer


class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

gamekey_platform/urls.py:

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet
```

```
router = DefaultRouter()
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
]
```

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
REST & DRF overview	15 min	What is REST? Resources, HTTP verbs (GET/POST/PATCH/DELETE), status codes. What does DRF add on top of Django? Serializers for data validation + transformation, ViewSets for resource operations, Routers for automatic URL generation.
Serializers	20 min	Write `GameSerializer` and `PublisherSerializer`. Explain *why* we exclude `webhook_secret` from the publisher serializer — never expose secrets in API responses. Show `fields = '__all__'` first, then demonstrate the risk.
ViewSets	20 min	Write `GameViewSet` and `PublisherViewSet`. Compare ViewSet to a regular Django view: a ViewSet is one class that handles list, create, retrieve, update, destroy automatically.
Router & URLs	15 min	Register ViewSets with the router. Show what URLs the router generates (`/api/games/`, `/api/games/{id}/`). Browse the DRF web UI.
Live testing	10 min	Use curl or Postman/httpie to hit `GET /api/games/`, `POST /api/games/`, `GET /api/games/1/`. Read responses together.
Q&A + checkpoint	10 min	Verify browsable API works.

💡 Key Concepts to Introduce

- **Serializer = translator** — converts a Django model instance into JSON (serialization) and JSON into a validated Python dict (deserialization). Not just for output — serializers also validate incoming data.
- **`ModelSerializer`** — the shortcut. Reads the model's fields and generates the serializer automatically. `fields = '__all__'` includes every field; `exclude = (...)` removes specific ones.
- **ViewSet vs APIView** — `APIView` maps one HTTP method to one method (`get`, `post`). A `ViewSet` maps the full CRUD lifecycle to one class (`list`, `create`, `retrieve`, `update`, `destroy`). The router wires these to URLs.
- **`DefaultRouter`** — generates `/api/games/` (list + create) and `/api/games/{pk}/` (retrieve + update + delete) automatically. Also adds a browsable root at `/api/`.
- **DRF browsable API** — great for development; shows a web form for POST requests. In production, disable it with `'DEFAULT_RENDERER_CLASSES': ['rest_framework.renderers.JSONRenderer']`.

⚠ Common Mistakes to Address

- **`'rest_framework'` not in `INSTALLED_APPS`** — the browsable API won't load, static CSS is missing. Common symptom: `DisallowedHost` or ugly unstyled pages.
- **Exposing sensitive fields** — students use `fields = '__all__'` on `Publisher` and wonder why `webhook_secret` appears in the response. This is a teaching moment: always be explicit about what you expose.
- **Forgetting to wire `include(router.urls)`** — `router.urls` is a list of URL patterns; it must be included in `urlpatterns`.
- **ViewSet queryset not set** — if `queryset` is missing, DRF raises `AssertionError` about `basename`. Always set `queryset` or override `get_queryset()`.

? Check for Understanding

"What does a serializer do with incoming data before it reaches the database?"

Answer: A serializer validates the structure and types of incoming data against defined field rules, checks for required fields, runs custom validation logic (e.g., checking if a value is valid), and parses the raw JSON input into a validated Python dictionary (`serializer.validated_data`).

"What's the difference between `GET /api/games/` and `GET /api/games/1/`? Which ViewSet methods handle each?"

Answer: `GET /api/games/` retrieves a list of all games and is handled by the `list` method of `GameViewSet`. `GET /api/games/1/` retrieves the details of a single game with ID 1 and is handled by the `retrieve` method of `GameViewSet`.

"Why might you want `fields = ('id', 'title', 'price')` instead of `'__all__'`?"

Answer: Explicitly defining fields improves security and API efficiency by ensuring that sensitive internal fields (like database flags, internal statuses, or relationship keys) are not leaked to the frontend, and reduces bandwidth by excluding unused fields.

"What would happen if we forgot to exclude `webhook_secret` from the publisher serializer?"

Answer: The `webhook_secret` would be serialized and returned in public GET/POST responses. This would allow anyone viewing the API output to obtain the secret, enabling them to forge webhook requests and make unauthorized deliveries appear authentic to the publisher's system.

✓ Day Checkpoint

`GET /api/games/` returns `200 OK` with a JSON array (possibly empty). `POST /api/games/` with a valid body creates a game and returns `201 Created`. Student confirms `webhook_secret` does not appear in `GET /api/publishers/` responses.